

Creative Programming: Data Representation

Lecture 7

Narration

Table of contents

- Debugging
- Objects
- Loop loop loop loop loop loop loop loop loop

Narration

One particle is removed per update
particles: 15 | mouse: add 100



Narration

(Re)visiting slides?

You can jump to any slide with the menu to the (bottom) left.

Did you miss a slide or want to revisit?

Open the narration tab while studying to get an explanation of difficult slides.



Part 2 of Creative Programming

- We are now officially in the “Data + Project” stage!!!
- One last assignment
 - To respect your time, this final assignment is shorter
 - **Heavily recommended** to go through the first part as debugging is critical for your project

Narration

But first!

- We have a few lectures left and some extra room for requests
- Please fill out the menti so we know can tailor the remaining lectures to your needs: <https://www.menti.com/> (code: 1234 5678)

Narration

Debugging

Narration

Why this matters

- Seeing your questions and assignments, debugging seems the most challenging part
- Let us spend some time on practicing debugging to prevent getting stuck with your project later



Narration

Debugging mindset

- Bugs are normal: every programmer debugs
- Avoid random changes; test one hypothesis at a time
- Keep changes small so you can isolate cause and effect

Narration

Practical debugging workflow

1. Reproduce the bug consistently
2. Reduce to a small example
3. Check console errors first
4. Log key variables
5. Fix one issue at a time
6. Re-test and remove temporary logs

Narration

Debugging Practice

Class Assignment: Debug Party

1. We will go through four slides with buggy sketches.
2. Per slide you will have 10 minutes to identify and fix the bug.
3. Use the debugging strategies we discuss to find the issue.



Narration

When there is no output at all

- First suspicion: JavaScript error stopped execution
- Open browser inspect tools and check the Console tab
- Look for red errors and line numbers
- Fix the first error first; later errors are often side effects

Narration

[1/5] Typical “no output” causes

- Missing `}` or `)` in a function
- Typo in function names, e.g. `backgroundnd()`
- Drawing white shapes on white background
- Canvas created outside `setup()` by mistake

```
1 // Debug challenge 1: no output.  
2 // Goal: show a moving circle  
3  
4 let x = 40;  
5  
6 ▾ function setup() {  
7   createCanvas(700, 400);  
8 }  
9  
10 ▾ function draw() {  
11   background(255);
```

Narration

[2/5] If visuals look wrong, inspect values

- Use `console.log()` to inspect variables while running
- Log before and after updates
- Start with the variable you think is wrong
- Keep logs focused (too many logs create noise)

```
1 // Debug challenge 2: inspect values.
2 // Goal: smooth horizontal motion with a readable speed.
3
4 let x = 60;
5 let speed = 2;
6
7 function setup() {
8   createCanvas(700, 400);
9 }
10
11 function draw() {
```

The circle should move steadily left to right.

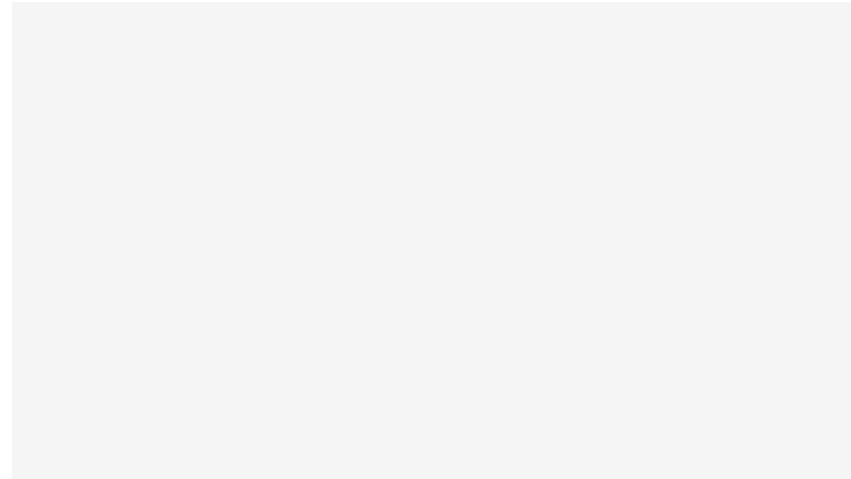


Narration

[3/5] Scope problems

- A common bug is when trying to access variables not in scope
- Variables in one function are not automatically available elsewhere
- If `draw()` needs a variable every frame, define it globally

```
1 // Debug challenge 3: scope bug.
2 // Goal: move player with arrow keys. Go diagonal when multiple are
  pressed.
3
4 let speed = 4;
5
6 ▾ function setup() {
7   createCanvas(700, 400);
8   let playerX = width * 0.5;
9   let playerY = height * 0.5;
10 }
11
12 ▾ function draw() {
```



Narration

[4/5] Loops and unexpected values

- Loop boundaries are common bug sources
- Check start, stop, and increment carefully
- Off-by-one errors are very common (< vs <=)
- Log loop variable `i` for first few iterations

```
1 // Debug challenge 4: Loop logic.
2 // Goal: draw exactly 10 same-size circles in one row.
3
4 ▾ function setup() {
5   createCanvas(700, 400);
6 }
7
8 ▾ function draw() {
9   background(248);
10
11   for (let i = 0; i < 10; i++) {
```

Expected: 10 circles of the same size.



Narration

[5/5] NaN and undefined bugs

- `undefined` means variable/value missing
- `NaN` means “not a number” (usually bad math input)
- Usually means a variable is not initialised or has the wrong value

```
1 // Debug challenge 5: NaN / undefined.
2 // Goal: dot follows the mouse smoothly.
3
4 let x = 350;
5 let y = 200;
6 let targetX;
7 let targetY;
8
9 ▾ function setup() {
10   createCanvas(700, 400);
11 }
12
13 ▾ function draw() {
```

The dot should follow both x and y mouse movement.

Narration

Objects

Narration

How have we stored data?

- Variables store a single value, arrays store lists
- Our data has always been structured attribute-wise, e.g. `xs[i]`, `ys[i]`, `speeds[i]`
- This can get confusing when we have many attributes for many items

Narration

Objects in the real world

- A bike has properties (color, speed, position)
- A bike has behaviours (accelerate, brake, turn)
- We can model many things this way: cars, elements of graphs, etc.



Creative Programming 2026 Lecture 7 - Ties Robroek - IT University of Copenhagen

Narration

A ball

- A ball has properties: `x`, `y`, `size`, `color`
- Instead of having these in arrays, we would like these to be grouped together
- This is what an object does: it groups related data together

Narration

Why not only variables/arrays?

- Variables and arrays are great, but parallel arrays can get confusing
- Example: `xs[i]`, `ys[i]`, `speeds[i]`, `sizes[i]`
- Objects keep all info for one entity in one place

```
1  let ballX = 120;  
2  let ballY = 200;  
3  let ballSize = 48;  
4  let ballSpeedX = 2.6;  
5  let ballSpeedY = 1.8;  
6  
7  function setup() {  
8    createCanvas(700, 400);  
9  }  
10  
11 function draw() {  
12   background(247);  
13
```

Single ball with loose variables



ballX: 684.20000000000035
ballSpeedX: 2.6

Narration

Objects in the code world

- In JavaScript, an object groups related data and behaviour
- This helps us keep code organized
- Especially useful when we have many similar things

```
1 ▾ let ball = { // Init an object
2   x: 120,
3   y: 200,
4   size: 48,
5   speedX: 2.6,
6   speedY: 1.8
7 };
8
9 ▾ function setup() {
10   createCanvas(700, 400);
11 }
12
13 ▾ function draw() {
```

Single object with properties



ball.x: 621.80000000000029
ball.speedX: -2.6

Narration

Objects in arrays

- We can make a factory function to create objects
- Then we can store many objects in an array
- This is a common pattern for games, simulations, and data visualizations

```
1 let balls = [];  
2  
3 // Factory function: creates one new ball object with random starting  
  values.  
4 function createBall() {  
5   return {  
6     x: random(50, width - 50),  
7     y: random(80, height - 50),  
8     size: random(20, 45),  
9     speedX: random(-2.5, 2.5),  
10    speedY: random(-2.5, 2.5),  
11    hue: random(180, 260)  
12  };
```



Narration

Methods on objects

- Objects can also contain functions, called methods
- Methods can directly use object data via `this`

```
1 ▾ function move() {  
2   // Apply current velocity to position each frame.  
3   this.x += this.vx;  
4   this.y += this.vy;  
5 }  
6  
7 ▾ function clampToCanvas() {  
8   // Keep the circle fully visible inside the canvas edges.  
9   this.x = constrain(this.x, this.size / 2, width - this.size / 2);  
10  this.y = constrain(this.y, this.size / 2, height - this.size / 2);  
11 }  
12  
13 ▾ let player = {  
14   x: 350,  
15   y: 220,  
16   vx: 0
```

Object methods using named functions
Arrow keys change velocity



Narration

1. Make an object, e.g. a flower or car, with a few properties (position, color, size).
2. Write a method to draw the object on the canvas.
3. With a factory, can you make an array of these objects with different properties?



**Loop loop loop loop loop
loop loop loop loop**

Narration

Loo(p)(k)ing back

- We have used for-loops for repetition
- Are there other ways to iterate over code?

```
1 // for loop
2 for (let i = 0; i < balls.length; i++) {
3   // update and draw balls[i]
4 }
```

Narration

Other kinds of loops

- `while` repeats as long as a condition is true
- `forEach` runs a function for each item in an array
- Both can be more readable for certain coding patterns, but they have their own pitfalls

Narration

The `while` loop

- Use `while` when you do not know how many retries you need
- The code will repeat as long as the condition holds true
- Always ensure the loop condition can become false

```
1 let circles = [];  
2 while (circles.length < 20) {  
3   // generation logic  
4   // will keep going until we have 20 circles  
5 }
```

Narration

while debugging checklist

- Frozen sketch? Suspect an infinite `while`
- Check the condition and the variables that must change

```
1 let circles = [];  
2  
3 ▾ function createCircle() {  
4 ▾   return {  
5     x: random(width),  
6     y: random(height),  
7     size: random(20, 60),  
8     r: random(80, 255),  
9     g: random(80, 255),  
10    b: random(80, 255)  
11  };  
12 }  
13  
14 ▾ function setup() {  
15   createCanvas(700, 400);  
16   noStroke();
```



Narration

forEach loops

- `forEach` is great when you want to run the same update/draw steps for every element
- It reads close to natural language: “for each elements, do this”
- Syntax is complicated: `array.forEach(element => { /* code */ });`

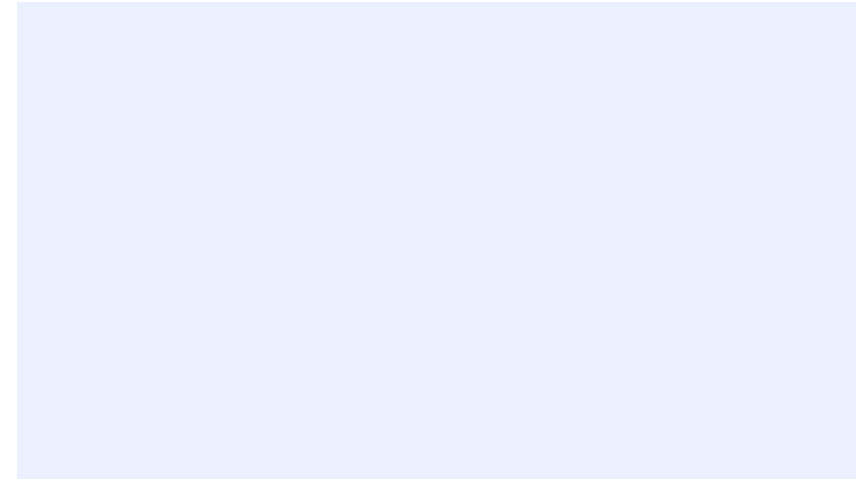
```
1 // Draw balls
2 balls.forEach(ball => {
3   circle(ball.x, ball.y, ball.size);
4 });
```

Narration

forEach example

- Works especially well with arrays of objects
- Be super careful with the syntax as there is a code block `{}` inside the parentheses `()`!

```
1  let bubbleArray = [];  
2  
3  function setup() {  
4    createCanvas(700, 400);  
5  }  
6  
7  function draw() {  
8    background(235, 244, 255);  
9  
10   // forEach: apply the same update + draw steps to every bubble.  
11   bubbleArray.forEach(bubble => {  
12     bubble.y -= bubble.speed;  
13  
14     // When a bubble leaves the top move it back to the bottom
```



Narration

Loop debugging

- Log only key values (`length`, `i`, one object sample)
- Test with a tiny array first (3-5 objects)
- Separate update and draw phases for clarity
- Keep loop bodies short and move repeated logic into functions

Narration

Loops

💡 Class Assignment: Loop Lab (`while` + `forEach`)

1. Create an array of 12+ visual objects (such as bubbles) using a `while` setup loop.
2. Update and draw them every frame using `forEach`.
3. Add a way to spawn more of the objects, e.g. key or mouse press.
4. Use debugging logs to check the number of objects and their properties as you go!



Narration